Search in Prolog, Scheduling in ASP

A* and IDA* for a labyrinth and the 8-puzzle
plus a Clingo tournament scheduler

Exam project — Artificial Intelligence and Laboratory
Logic Programming and Answer Set Programming module · 31 October 2026

Two activities, one methodology

Activity 1 — Prolog

- generic A* and IDA* engines;
- labyrinth with multiple exits;
- 8-puzzle with parity precheck;
- benchmark with frozen CSV data.

Activity 2 — ASP

- one round of a Pokémon League;
- dynamic arrivals and BYE handling;
- no repeated pair or BYE;
- lexicographic score balancing.

Common thread: a declarative problem statement, an independent correctness check, and a reproducible experiment.

A*: best-first search with an admissible estimate

$$f(n) = g(n) + h(n)$$

- frontier ordered by f;
- ties prefer larger g;
- goal tested on **pop**, not generation;
- optimal with an admissible h, even if inconsistent.

Implementation

- `library(heaps)`, key `f(F, -G, Seq)`;
- `library(assoc)` for best g and parents;
- no decrease-key ⟶ lazy deletion;
- cheaper path to a closed state ⟶ **reopen**.

IDA*: linear memory, repeated work

- first threshold: $h(\text{start})$;
- depth-first search cuts every f above the bound;
- next bound: smallest f that was cut off;
- path-only cycle checking $\longrightarrow$ space $O(d)$;
- states reached by another branch are expanded again.

The trade-off is the experiment: how much time does IDA* pay for the memory it saves, and how does the heuristic change that price?

A small contract keeps the engine domain-agnostic

Engine (`prolog/search/`)
- `a_star/4`, `ida_star/4`;
- options and statistics;
- no domain imports.

Domain (`prolog/domains/`)
- `initial_state/2`, `goal_test/2`;
- `successors/3`, `heuristic/3`;
- `state_key/2`, `state_label/2`.

`Problem = problem(DomainModule, Config)` — the same engines run the labyrinth and the 8-puzzle, and the tests can plug in hand-built graphs.

Two domains, four required heuristics

Labyrinth

- `pos(Row, Col)`, four orthogonal moves;
- Manhattan and Euclidean, minimum over all exits;
- both admissible and consistent;
- unreachable exit only lowers the estimate.

default 10×10 (cost 16), multi-exit 9×9 (12), open 8×8 (7).

8-puzzle

- flat nine-element state, `0` is the blank;
- inversion parity decides solvability;
- misplaced tiles and Manhattan;
- Manhattan dominates misplaced tiles.

12 solvable instances, optimal cost 2–16.

Correctness is checked against independent oracles

- 46 Prolog tests: 11 engine, 18 labyrinth, 17 puzzle;
- breadth-first distance oracle, not the A*/IDA* code;
- every returned path is replayed through the domain contract;
- hand-built graphs force lazy deletion and reopening;
- heuristic admissibility and consistency on **every** free labyrinth cell;
- parity rejects unsolvable puzzles before any search.

A* and IDA* agree on every optimal cost; the oracle confirms it independently.

Benchmark: 60 configurations, 5 repetitions, 300 runs

Matrix

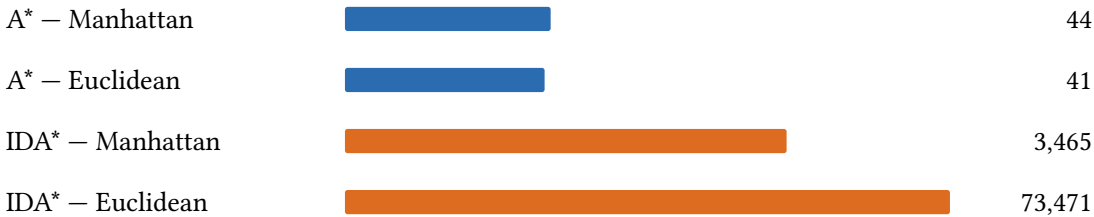
- 3 labyrinth maps × 2 heuristics × 2 solvers;
- 12 puzzle instances × 2 heuristics × 2 solvers;
- 60 s timeout; GC before each run.

Output

- `runs.csv`: one row per repetition;
- `summary.csv`: median counters;
- frozen in the repository;
- whole run ≈ 14 s.

Peak RSS is intentionally absent: `statistics/2` reports current, not peak, memory; frontier size and maximum depth describe the memory shape.

Labyrinth: the heuristic matters far more for IDA*



Default 10×10 map, median expanded nodes, logarithmic bars. All four runs return the same optimal cost 16. A*: 44 vs 41 (tie-breaking decides); IDA*: 3,465 vs 73,471 (weaker estimate multiplies thresholds).

8-puzzle: stronger heuristic, fewer expansions

Instance	A* misplaced	A* Manhattan	Cost	IDA* Manhattan
easy_8	9	9	8	14
medium_12	64	22	12	87
medium_15	355	63	15	165
medium_16	341	48	16	180

All 12 instances: A* Manhattan 323 expansions, A* misplaced 1,577, IDA* Manhattan 799, IDA* misplaced 7,550. On `medium_16` Manhattan cuts A* by 7× and IDA* by 13× at the same cost.

The trade-off, stated honestly

A*

- far fewer expansions;
- keeps frontier, best-g map, parents;
- memory grows with distinct states.

IDA*

- linear memory;
- revisits states through other branches;
- can expand 1,670× more nodes (default map, Euclidean).

No universal winner: report both the frontier size and the expansion counts, and choose according to the memory budget.

ASP: one round of a Pokémon League

Input

- active players and scores in wins;
- cumulative `played/2` and `had_bye/1` history;
- optional `new_player/1` arrivals.

Rules

- one opponent per player; one BYE on an odd field, none on an even one;
- no repeated pair or BYE; nobody withdraws; a BYE is a win.

One round is a polynomial matching problem; ASP is used for declarative constraints and built-in optimisation, not for NP-hardness.

Generate–test–optimize in Clingo

```
candidate(X, Y) :- player(X), player(Y), X < Y,
                   not played(X, Y), not played(Y, X).
{match(X, Y) : candidate(X, Y)}.
incident(P, Q) :- match(P, Q).
incident(P, Q) :- match(Q, P).
:- player(P), 2 { incident(P, Q) : player(Q) }.
bye(P) :- player(P), not has_match(P).
```

- `X < Y`: canonical pairs; `incident/2`: both orientations;
- `bye/1` is output, `had_bye/1` is history;
- integrity constraints reject repeats and malformed input.

Lexicographic objective, with a sum baseline

```
#minimize{1@3, X, Y : unequal_pair(X, Y)}.   % most equal-score pairs
#minimize{D@2, X, Y : diff(X, Y, D)}.        % smallest total gap
#minimize{S@1, P : bye_score(P, S)}.         % fairest BYE
```

Model	Pairings	Equal	Total gap
official	`a–e, b–f, c–d`	1	5
sum	`a–c, b–d, e–f`	0	3

Scores $0, 0, 1, 1, 2, 3$ with `played(a,b)`: maximising equal pairs is not the same as minimising the total difference.

Validation and multiple rounds

Independent validation

- canonical pairs and active players;
- one opponent per player;
- correct BYE count;
- no repeated pair or BYE;
- full field coverage.

Driver

- `--model official|sum`;
- `--win-points` (default 1);
- `--advance` writes the next instance;
- UNSAT: relaxation ladder and diagnosis.

Round 1: `alice–bob, carol–dave`, BYE erin, cost `1,1,0`. Round 2: `alice–frank, bob–carol, dave–erin`, no BYE, `1,4,0`. Round 3: `bob–dave, carol–erin, frank–grace`, BYE alice, `2,4,5`.

Conclusion

- generic A*/IDA* engine with a six-predicate domain contract;
- two domains, four heuristics, independent breadth-first oracles;
- 46 Prolog tests and 21 Python tests, all green in CI;
- lexicographic ASP model plus a sum baseline and a concrete counterexample;
- driver validates invariants, advances rounds, and diagnoses UNSAT;
- frozen CSV data and a fully reproducible command set.

Questions?